

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 1	CLOs to be covered: CLO1
Lab Title: I/O, Data types, and conditional statements in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

Lab Goals/Objectives:

- 1- Learn about Input and Output commands of Python
- 2- Know about the data types in python
- 3- Do decision control in python

Equipment Required: Computer system with Pycharm IDE and python package installed on it

Theory:

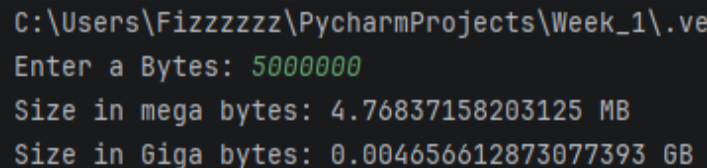
Python uses `input()` to take user input (always as a string) and `print()` to display output, with data types like `int`, `float`, `str`, and `bool` used to store different kinds of values. Operators — arithmetic (+, -, *, /), relational (==, >, <), and logical (and, or, not) — are used to perform calculations and comparisons. Conditional statements (if, elif, else) allow a program to execute different code blocks based on whether a condition is true or false.

Lab Work:

Task 1:

```
bytes = float(input("Enter a Bytes: "))
megabytes = bytes/(1024 * 1024)
gigabytes = bytes / (1024 * 1024 * 1024)
print("Size in mega bytes:", megabytes, "MB")
print("Size in Giga bytes:", gigabytes, "GB")
```

Result:

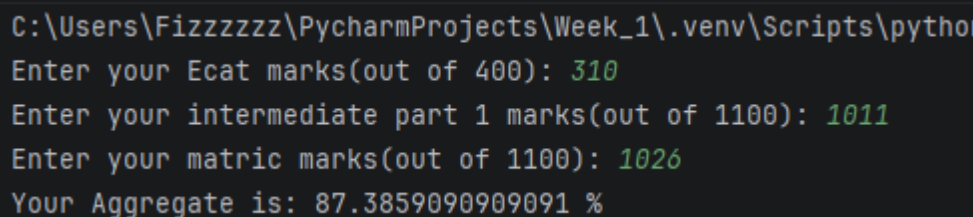
A terminal window showing the execution of the Task 1 program. The prompt is 'C:\Users\Fizzzzzz\PycharmProjects\Week_1\.venv\Scripts\python'. The user enters '5000000'. The output shows 'Size in mega bytes: 4.76837158203125 MB' and 'Size in Giga bytes: 0.004656612873077393 GB'.

```
C:\Users\Fizzzzzz\PycharmProjects\Week_1\.venv\Scripts\python
Enter a Bytes: 5000000
Size in mega bytes: 4.76837158203125 MB
Size in Giga bytes: 0.004656612873077393 GB
```

Task 2:

```
ecat = float(input("Enter your Ecat marks(out of 400): "))
inter = float(input("Enter your intermediate part 1 marks(out of 1100): "))
matric = float(input("Enter your matric marks(out of 1100): "))
ecat_percent = (ecat / 400) * 100
inter_percent = (inter / 1100) * 100
matric_percent = (matric / 1100) * 100
aggregate = (ecat_percent * 0.33) + (inter_percent * 0.50) + (matric_percent * 0.17)
print("Your Aggregate is:", aggregate, "%")
```

Result:

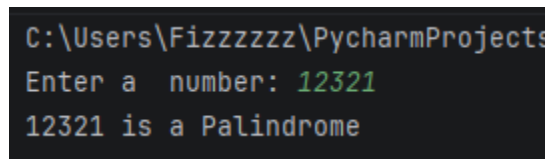
A terminal window showing the execution of the Task 2 program. The prompt is 'C:\Users\Fizzzzzz\PycharmProjects\Week_1\.venv\Scripts\python'. The user enters '310', '1011', and '1026'. The output shows 'Your Aggregate is: 87.3859090909091 %'.

```
C:\Users\Fizzzzzz\PycharmProjects\Week_1\.venv\Scripts\python
Enter your Ecat marks(out of 400): 310
Enter your intermediate part 1 marks(out of 1100): 1011
Enter your matric marks(out of 1100): 1026
Your Aggregate is: 87.3859090909091 %
```

Task 3:

```
number = input("Enter a number: ")
reverse = number[::-1]
if number == reverse:
    print(number,"is a Palindrome")
else:
    print(number,"is Not a Palindrome")
```

Result:

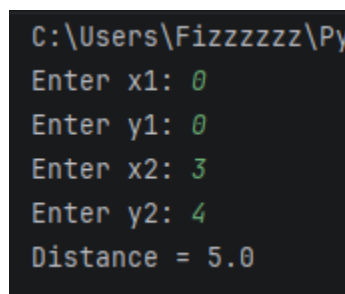


```
C:\Users\Fizzzzzz\PycharmProjects
Enter a number: 12321
12321 is a Palindrome
```

Task 4:

```
import math
x1 = float(input("Enter x1: "))
y1 = float(input("Enter y1: "))
x2 = float(input("Enter x2: "))
y2 = float(input("Enter y2: "))
distance = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
print("Distance =", distance)
```

Result:



```
C:\Users\Fizzzzzz\Py
Enter x1: 0
Enter y1: 0
Enter x2: 3
Enter y2: 4
Distance = 5.0
```

Task 5:

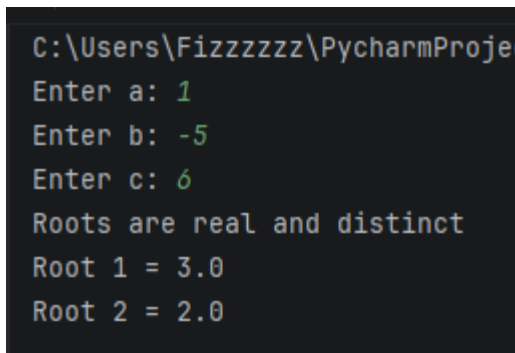
```
import math
a = float(input("Enter a: "))
b = float(input("Enter b: "))
c = float(input("Enter c: "))
d = (b * b) - (4 * a * c)
```

```

if d == 0:
    print("Roots are real and equal")
elif d > 0:
    print("Roots are real and distinct")
else:
    print("Roots are imaginary")
if d >= 0:
    root1 = (-b + math.sqrt(d)) / (2 * a)
    root2 = (-b - math.sqrt(d)) / (2 * a)
    print("Root 1 =", root1)
    print("Root 2 =", root2)

```

Result:



```

C:\Users\Fizzzzzz\PycharmProje
Enter a: 1
Enter b: -5
Enter c: 6
Roots are real and distinct
Root 1 = 3.0
Root 2 = 2.0

```

Conclusions:

This lab helped me understand input/output operations, data types, operators, and conditional statements in Python. I implemented programs for unit conversion, palindrome checking, distance calculation, and finding roots of a quadratic equation, which strengthened my understanding of how conditions and operators are used to solve real-world problems.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 2	CLOs to be covered: CLO1
Lab Title: Loops in python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 2

Lab Goals/Objectives:

- To understand the concept of loops in Python
- To know about how to use for-loop in python

Equipment Required: Computer system with Pycharm IDE and python package installed on it

Theory:

A loop repeats a block of code without rewriting it. A **while** loop runs as long as its condition stays true. A **for** loop iterates a fixed number of times using **range()**. A nested loop is a loop inside another loop, used for tasks like patterns or grids. An infinite loop occurs when a **while** loop's condition never becomes false.

Lab Work:

Task 1:

```
binary = input("Enter a binary number: ")
```

```
decimal = 0
```

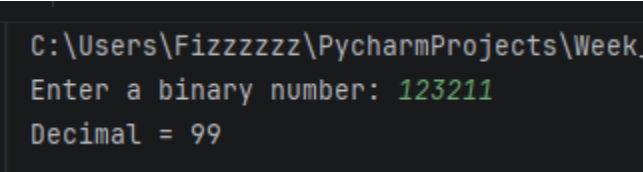
```
power = 0
```

```
for digit in binary[::-1]:
```

```
    decimal = decimal + int(digit) * (2 ** power)
```

```
power = power + 1  
  
print("Decimal =", decimal)
```

Result:

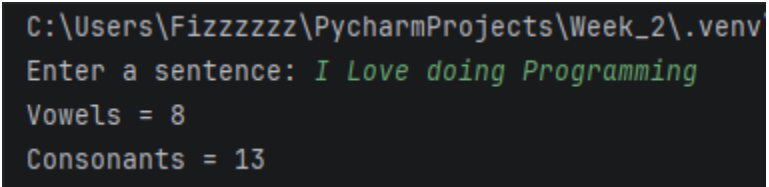
A terminal window with a dark background. The first line shows the file path 'C:\Users\Fizzzzzz\PycharmProjects\Week...'. The second line is a prompt 'Enter a binary number:' followed by the user input '123211' in green. The third line shows the output 'Decimal = 99' in white.

```
C:\Users\Fizzzzzz\PycharmProjects\Week...  
Enter a binary number: 123211  
Decimal = 99
```

Task 2:

```
sentence = input("Enter a sentence: ")  
  
vowels = 0  
consonants = 0  
  
for ch in sentence.lower():  
    if ch.isalpha():  
        if ch in "aeiou":  
            vowels = vowels + 1  
        else:  
            consonants = consonants + 1  
  
print("Vowels =", vowels)  
print("Consonants =", consonants)
```

Result:

A terminal window with a dark background. The first line shows the file path 'C:\Users\Fizzzzzz\PycharmProjects\Week_2\.venv'. The second line is a prompt 'Enter a sentence:' followed by the user input 'I Love doing Programming' in green. The third line shows the output 'Vowels = 8' in white. The fourth line shows the output 'Consonants = 13' in white.

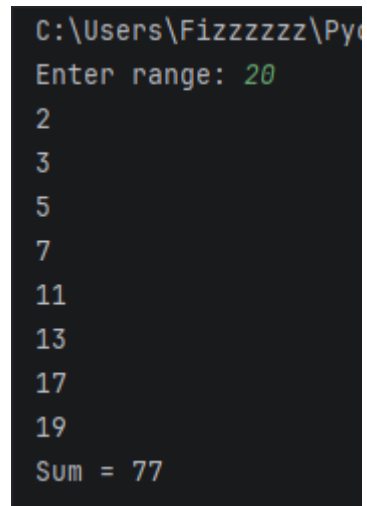
```
C:\Users\Fizzzzzz\PycharmProjects\Week_2\.venv  
Enter a sentence: I Love doing Programming  
Vowels = 8  
Consonants = 13
```

Task 3:

```
num = int(input("Enter range: "))  
  
total = 0  
  
for i in range(2, num + 1):
```

```
prime = True
for j in range(2, i):
    if i % j == 0:
        prime = False
        break
if prime:
    print(i)
total = total + i
print("Sum =", total)
```

Result:



```
C:\Users\Fizzzzzz\Pyo
Enter range: 20
2
3
5
7
11
13
17
19
Sum = 77
```

Task 4:

“for i in range(length):

password += random.choice(characters)”

- This is the core line — it randomly picks one character from the `characters` pool and adds it to the password, repeated by the for loop until the password reaches the desired length

Result:

```

C:\Users\Fizzzzzz\PycharmProjects\Week_2\.venv
Enter password length: 21
Include uppercase? (yes/no): yes
Include lowercase? (yes/no): yes
Include digits? (yes/no): no
Include special characters? (yes/no): no
generate Password: yAHEQDvYrFQWhYcSQns0g

```

Task 5:

```

for i in range(1, 5):
    for j in range(i):
        print("", end=" ")
    print()
    for i in range(3, 0, -1):
        for j in range(i):
            print("", end=" ")
        print()
    print()

```

Result:

```

C:\Users\Fizzzzzz\
*
* *
* * *
* * * *
* * *
* *
*

```

Conclusions:

This lab helped me understand the use of `for` and nested loops in Python. I implemented programs for binary-to-decimal conversion, vowel/consonant counting, prime number generation, password generation, and star patterns, which strengthened my understanding of how loops are used to repeat and control program logic.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 3	CLOs to be covered: CLO1
Lab Title: While Loop	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 3

Lab Goals/Objectives:

- To understand the concept of while loops in Python
- To learn how to use a while loop to repeat a block of code based on a condition
- To know how to control loop execution using break and continue statements
- To understand the difference between finite and infinite while loops

Equipment Required: Computer system with Pycharm IDE and python package installed on it

Theory:

A **while loop** is a control structure that repeatedly executes a block of code as long as a given condition is **True**. It is useful when the number of iterations is not known beforehand. The loop checks the condition before each iteration and stops automatically when the condition becomes **False**. Properly updating the loop variable prevents an infinite loop.

Lab Work:

Task 1:

```
total_students = int(input("enter total number of students : "))

val = 1

while val <= total_students:

    student_name = input("enter the student name : ")

    roll_number = input("enter your roll number : ")

    obtained_grades = int(input("enter your obtained grade: "))

    total_grades = int(input("enter your total grade: "))

    percentage = (obtained_grades/total_grades)*100

    print("Your percentage is : ",percentage)

    if percentage > 90 :

        grade = ("A")

    elif percentage > 85 :

        grade = ("A-")

    elif percentage > 80 :

        grade = ("B+")

    elif percentage > 75 :

        grade = ("B")

    elif percentage > 70 :

        grade = ("B-")

    elif percentage > 65 :

        grade = ("C+")

    elif percentage > 60 :
```

```
grade = ("C")
elif percentage > 55 :
grade =("C-")
elif percentage > 50 :
grade =("D")
else:
grade = ("F")
print(grade)

print(f'{student_name} of roll number {roll_number} have obtained percentage {percentage}
with {grade} grade")

val = val+1
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProjects\Week 8\.venv\Scripts\python.exe"
enter total number of students : 1
enter the student name : Fizza
enter your roll number : 14
enter your obtained grade: 85
enter your total grade: 100
Your percentage is : 85.0
B+
Fizza of roll number 14 have obtained percentage 85.0 with B+ grade
```

Conclusions:

This program uses a **while loop** to process the records of multiple students efficiently. It calculates each student's percentage, determines the corresponding grade based on predefined criteria, and displays the results. The program demonstrates the practical use of loops, conditional statements, user input, and arithmetic operations in Python.

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 4	CLOs to be covered: CLO1
Lab Title: Nested Loop	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 4

Lab Goals/Objectives:

- To understand the concept of nested loops in Python.
- To learn how to use one loop inside another to solve programming problems.
- To implement nested loops for pattern printing and repetitive tasks.

Equipment Required: Computer system with Pycharm IDE and python package installed on it

Theory:

A **nested loop** is a loop placed inside another loop. The inner loop executes completely for each iteration of the outer loop. Nested loops are commonly used for pattern printing, working with rows and columns, and solving problems that require repeated iterations within another loop.

Lab Work:

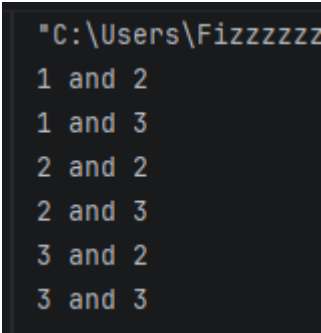
Task 1:

```
for i in range(1, 4):
```

```
    for j in range(2, 4):
```

```
        print(f'{i} and {j}')
```

Result:



```
"C:\Users\Fizzzzzzz
1 and 2
1 and 3
2 and 2
2 and 3
3 and 2
3 and 3
```

Task 2:

```
for i in range(1, 10):
    for j in range(1, i+1):
        print("*", end=" ")

stars = 10
for i in range(stars):
    spaces = stars - i - 1
    print(' ' * spaces + "*" * (2 * i + 1))
```

Result:

```
"C:\Users\Fizzzzzz\Pycharm
* * * * *
***
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

Task 3:

```
for i in range(1, 11):
    for j in range(1, 5):
        print(i*j , end = "\t")
    print()
```

Result:

```
"C:\Users\Fizzzzzz\Py
1  2  3  4
2  4  6  8
3  6  9  12
4  8  12 16
5  10 15 20
6  12 18 24
7  14 21 28
8  16 24 32
9  18 27 36
10 20 30 40
```

Task 4:

```
for i in range(1, 5):
    for j in range(1, i+1):
        print("*", end = " ")
    print()
```

Result:

```
"C:\Users\Fi
*
**
***
****
```

Task 5:

```
for i in range(1, 5):
    for j in range(4-i):
        print(" ", end = "")
    for j in range(2*i+1):
        print("*", end = "")
    print()
```

Result:

```
"C:\Users\Fizzz
    ***
   *****
  *********
 **********
 **********
```

Task 6:

```
for i in range(4):
    for j in range(4):
        print("*", end= " ")
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProjects\W
* * * * *
* * * * *
```

Task 7:

```
for i in range(1, 6):
    for j in range(i):
        print("*", end= " ")
    print()
```

Result:



```
"C:\Users\Fizzz
*
* *
* * *
* * * *
* * * * *
```

Task 8:

```
for i in range(4):
    for j in range(4):
        print("*", end= " ")
    print()
```

Result:



```
"C:\Users\Fizzz
* * * *
* * * *
* * * *
* * * *
```

Conclusion:

This lab helped in understanding the concept of **nested loops** in Python and their practical applications. By using loops inside other loops, different patterns and multiplication tables were generated efficiently. The lab improved understanding of loop execution and strengthened problem-solving skills through repeated iterations.

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 5	CLOs to be covered: CLO2
Lab Title: Functions in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 5

Lab Goals/Objectives:

- To understand the concept of user defined functions in Python
- To learn how to define and call functions in Python
- To understand anonymous function in Python
- To know about doc strings in python

Equipment Required: Computer system with Pycharm IDE and python package installed on it

Theory:

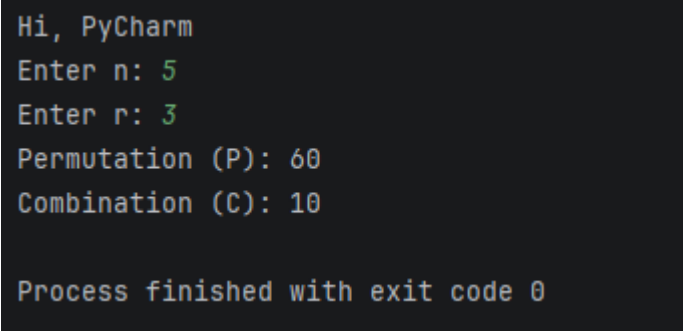
A function is a reusable block of code defined using the `def` keyword that performs a specific task and can be called multiple times. Functions can take parameters as input and use the `return` statement to send back a result, which helps avoid repeating code. Python also supports `lambda` functions, which are short, one-line anonymous functions used for simple operations. Using user-defined functions makes programs more organized, reusable, and easier to read and maintain.

Lab Work:

Task 1:

```
def factorial(num):  
    fact = 1  
    for i in range(1, num + 1):  
        fact = fact * i  
    return fact  
  
def permutation(n, r):  
    return factorial(n) // factorial(n - r)  
  
def combination(n, r):  
    return factorial(n) // (factorial(r) * factorial(n - r))  
  
n = int(input("Enter n: "))  
r = int(input("Enter r: "))  
print("Permutation =", permutation(n, r))  
print("Combination =", combination(n, r))
```

Result:



```
Hi, PyCharm  
Enter n: 5  
Enter r: 3  
Permutation (P): 60  
Combination (C): 10  
  
Process finished with exit code 0
```

Task 2:

```
large = lambda a, b: a if a > b else b

def table(num):

    limit = int(input("Enter range: "))

    for i in range(1, limit + 1):

        print(num, "x", i, "=", num * i)

num1 = int(input("Enter first number: "))

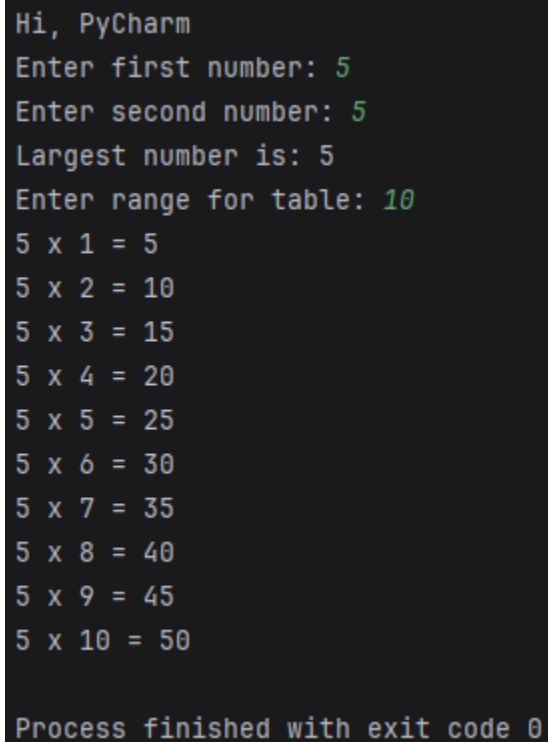
num2 = int(input("Enter second number: "))

big = large(num1, num2)

print("Larger number is:", big)

table(big)
```

Result:



```
Hi, PyCharm
Enter first number: 5
Enter second number: 5
Largest number is: 5
Enter range for table: 10
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

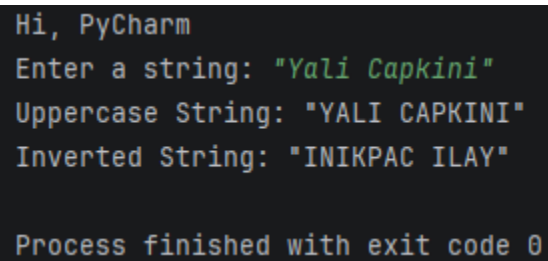
Process finished with exit code 0
```

Task 3:

```
upper = lambda text: text.upper()
```

```
def invert(word):  
    print(word[::-1])  
  
text = input("Enter a string: ")  
  
result = upper(text)  
  
print("Uppercase:", result)  
  
invert(result)
```

Result:



```
Hi, PyCharm  
Enter a string: "Yali Capkini"  
Uppercase String: "YALI CAPKINI"  
Inverted String: "INIKPAC ILAY"  
  
Process finished with exit code 0
```

Task 4:

```
def celsius_to_fahrenheit(c):  
    return (c * 9/5) + 32  
  
def fahrenheit_to_celsius(f):  
    return (f - 32) * 5/9  
  
c = float(input("Enter Celsius: "))  
f = float(input("Enter Fahrenheit: "))  
  
print("Fahrenheit =", celsius_to_fahrenheit(c))  
    print("Celsius =", fahrenheit_to_celsius(f))
```

Result:

```
Hi, PyCharm
Enter temperature in Fahrenheit: 45
Temperature in Celsius: 7.222222222222222
Enter temperature in celsius:50
Temperature in Fahrenheit: 122.0

Process finished with exit code 0
```

Task 5:

```
def calculate_gpa():
    subjects = int(input("Enter number of subjects: "))
    total_points = 0
    total_credit_hours = 0
    for i in range(subjects):
        gp = float(input("Enter grade point: "))
        ch = int(input("Enter credit hours: "))
        total_points = total_points + (gp * ch)
        total_credit_hours = total_credit_hours + ch
    gpa = total_points / total_credit_hours
    return gpa
print("GPA =", calculate_gpa())
```

Result:

```

Hi, PyCharm
Enter number of subjects: 5
Enter grade point: 3.5
Enter credit hours: 3
Enter grade point: 3.0
Enter credit hours: 3
Enter grade point: 4.0
Enter credit hours: 3
Enter grade point: 3.7
Enter credit hours: 3
Enter grade point: 3.8
Enter credit hours: 3
Your GPA is: 3.6

Process finished with exit code 0

```

Conclusions:

This lab helped me understand how to define and use user-defined functions in Python, including how parameters and return values work. I implemented programs for permutation/combination calculation, temperature conversion, and GPA calculation, which strengthened my understanding of how functions help organize and reuse code effectively.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 6	CLOs to be covered: CLO2
Lab Title: Local and global variable in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 6

Lab Goals/Objectives:

- To understand the difference between local and global variables in Python.
- To learn how variable scope affects program execution.
- To use local and global variables correctly in functions.

Equipment Required: Computer system with Pycharm IDE and python package installed on it

Theory:

A **local variable** is declared inside a function and can only be accessed within that function. A **global variable** is declared outside all functions and can be accessed throughout the program. Understanding variable scope helps in writing organized and error-free Python programs.

Lab Work:

Task 1:

```
x = "45"
y = "fizza"
print(x+y)

def a():
    print(x)
```

```
print(globals()["x"])  
a()
```

Result:

```
C:\Users\Fizz  
45fizza  
45  
45
```

Task 2:

```
x = "45"  
y = "fizza"  
print(x+y)  
  
def a():  
    print(x)  
a()
```

Result:

```
C:\Users\Fiz  
45fizza  
45
```

Conclusion:

This lab helped in understanding the difference between **local** and **global** variables in Python. It demonstrated how global variables can be accessed inside functions, while local variables are limited to the function in which they are declared. Understanding variable scope helps in writing organized, efficient, and error-free programs.

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 7	CLOs to be covered: CLO3
Lab Title: List, Tuple in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 7

Lab Goals/Objectives:

- To understand the concept of lists and tuples in Python
- To learn how to create, access, and modify list elements
- To understand the difference between lists (mutable) and tuples (immutable)
- To apply list and tuple operations to solve real-world problems

Equipment Required: Computer system with Pycharm IDE and python package installed on it

Theory:

A list in Python is an ordered, mutable collection of items enclosed in square brackets [], meaning its elements can be changed after creation. A tuple is an ordered, immutable collection enclosed in parentheses (), meaning its elements cannot be modified once created. Both support indexing (starting from 0), slicing, and iteration using loops. Lists support methods like append(), remove(), pop(), and sort(), while tuples are used when data should remain constant throughout the program.

Lab Work:

Task 1:

```
students = ["Fizza", "Ayfa", "Tehreem", "Sara"]
print("Original list:", students)
print("First student:", students[0])
students.append("Zara")
print("After append:", students)
students.remove("Sara")
print("After remove:", students)
for name in students:
    print(name)
```

Result:

```
Original list: ['Fizza', 'Ayfa', 'Tehreem', 'Sara']
First student: Fizza
After append: ['Fizza', 'Ayfa', 'Tehreem', 'Sara', 'Zara']
After remove: ['Fizza', 'Ayfa', 'Tehreem', 'Zara']
Fizza
Ayfa
Tehreem
Zara
```

Task 2:

```
courses = ("Python", "Calculus", "Physics", "ICT")
print("Courses tuple:", courses)
print("First course:", courses[0])
print("Last course:", courses[-1])
print("Number of courses:", len(courses))
for course in courses:
    print(course)
```

Result:

```
Courses tuple: ('Python', 'Calculus', 'Physics', 'ICT')
First course: Python
Last course: ICT
Number of courses: 4
Python
Calculus
Physics
ICT
```

Conclusion:

This lab helped me understand the difference between lists and tuples in Python. Lists are mutable and support methods like `append()` and `remove()`, while tuples are immutable and used when data should not change. I learned how to create, access, modify, and iterate over both data structures to solve real-world problems.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 8	CLOs to be covered: CLO3
Lab Title: Dictionary and sets in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 8

Lab Goals/Objectives:

- To understand the concept of sets and dictionaries in Python
- To learn how to create and manipulate sets using built-in methods
- To understand set operations like union, intersection, and difference
- To learn how to store and retrieve data using dictionaries

Equipment Required:

Computer system with PyCharm IDE and Python package installed on it.

Theory:

A set in Python is an unordered collection of unique elements enclosed in curly braces {}. Sets automatically remove duplicates and support operations like union, intersection, and difference. A dictionary is an ordered collection of key-value pairs also enclosed in curly braces {}, where each key must be unique. Dictionaries allow fast data lookup using keys, and support methods like items(), keys(), values(), and setdefault(). Both sets and dictionaries are widely used for data organization and manipulation.

Lab Work:

Task 1:

```
a = set([1,2,3,4])
print(a)
print(type(a))
```

Result:

```
"C:\Users\Fizzzzz\PycharmProjects\Week1\Task1\Task1.py"
{1, 2, 3, 4}
<class 'set'>
```

Task 2:

```
S = {"CE", "CS", "Mathematics", "CE", "DS", "SE"}
for i in S:
    print(i)
if "CE" in S:
    print("yes")
```

Result:

```
"C:\Users\Fizzzzz\PycharmProjects\Week1\Task1\Task1.py"
DS
Mathematics
CS
CE
SE
yes
```

Task 3:

```
S = {"CE", "CS", "Mathematics", "CE", "DS", "SE"}
S.discard("CS")
print(S)
```

Result:

```
"C:\Users\Fizzzzzzz\PycharmProjects\Week1\Task1\Task1.py"
{'Mathematics', 'DS', 'CE', 'SE'}
```

Task 4:

```
S = {"CE", "CS", "Mathematics", "CE", "DS", "SE"}
S.remove("CE")
print(S)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProjects\Week 11\venv\Scripts\python.exe" "C:\Users\Fizzzzzz\PycharmProjects\Week 11\task 4.py"
{'CS', 'SE', 'DS', 'Mathematics'}
```

Task 5:

```
S = {"CE", "CS", "Mathematics", "CE", "DS", "SE"}
S.remove("AI")
print(S)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProjects\Week 11\venv\Scripts\python.exe" "C:\Users\Fizzzzzz\PycharmProjects\Week 11\task 5.py"
Traceback (most recent call last):
  File "C:\Users\Fizzzzzz\PycharmProjects\Week 11\task 5.py", line 2, in <module>
    S.remove("AI")
    ~~~~~~^~~~~~
KeyError: 'AI'
```

Task 6:

```
S = {"CE", "CS", "Mathematics", "CE", "DS", "SE"}
S.pop()
print(S)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProjects\Week 11\venv\Scripts\python.exe" "C:\Users\Fizzzzzz\PycharmProjects\Week 11\task 6.py"
{'Mathematics', 'CS', 'CE', 'SE'}
```

Task 7:

```
S = {"CE", "CS", "Mathematics", "CE", "DS", "SE"}
S.clear()
print(S)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmPro
set()
```

Task 8:

```
S1 = {"Ali", "Ayesha", "Nawal"}
S2 = {12, 13, 19}
S3 = {"1", "cat", (1,2,3)}
S1.update(S2)
print(S2)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmPro
{19, 12, 13}
```

Task 9:

```
S1 = {"UET"}
S2 = {"UET"}
S3 = {"1", "cat", (1,2,,)}
S1.intersection(S2)
print(S1)
```

Result:

```
"C:\Users\Fizzzzzz\Pych
{'UET'}
```

Task 10:

```
S1 = {"Ali", "Ayesha", "Fatima", 90}
S2 = {90, 70, 80}
S3 = {1, "cat", "Ali33"}
S1.intersection_update(S2)
print(S1)
```

Result:

```
"C:\Users\Fizzzzzz\Pychar
{90}
```

Task 11:

```
S1 = {"Ali", "Ayesha", "Fatima", 90}
S2 = {90, 70, 80}
s = S1.symmetric_difference(S2)
print(s)
print(S1)
print(S2)
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\Week
{70, 'Ayesha', 80, 'Ali', 'Fatima'}
{'Fatima', 90, 'Ali', 'Ayesha'}
{80, 90, 70}
```

Task 12:

```
S1 = {"Ali", "Ayesha", "Fatima", 90}
S2 = {90, 70, 80}
s = S1.difference(S2)
print(s)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProject
{'Ayesha', 'Ali', 'Fatima'}
```

Task 13:


```
S1 = {1,2,3,4}
S2 = {"a,b,c,d"}
s = S1.isdisjoint(S2)
print(s)
```

Result:

```
"C:\Users\Fizzzzzz\Pychar
True
Process finished with exit
```

Task 14:

```
s1 = {1,2,3,4,5}
s2 = {2,3,4,5,6,7,8}
print(s2.issubset(s1))
```

Result:

```
"C:\Users\Fizzzzzz\Pyc
False
```

Task 15:

```
fruits = {
    "Apple": "سیب",
    "Mango": "آم",
    "Banana": "کیلا",
    "Grapes": "انگور",
    "Orange": "نارنگی"
}
for english, urdu in fruits.items():
    print(english, "=", urdu)
```

Result:

```
"C:\Users\Fizzzzzz\Pycha
Apple = سیب
Mango = آم
Banana = کیلا
Grapes = انگور
Orange = نارنگی
```

Conclusion:

This lab helped me understand sets and dictionaries in Python. I learned how sets automatically remove duplicates and support mathematical operations like intersection and difference, while dictionaries store data as key-value pairs for fast lookup. These data structures are essential for organizing and manipulating data efficiently.

LAB No 9

- VIVA

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 10	CLOs to be covered: CLO2
Lab Title: Recursion in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 10

Lab Goals/Objectives:

- To understand the concept of recursion in Python
- To learn how to define a recursive function with a base case and recursive case
- To apply recursion to solve mathematical problems like factorial and Fibonacci
- To understand how recursive function calls work on the call stack

Equipment Required: Computer system with PyCharm IDE and Python package installed on it.

Theory:

Recursion is a technique where a function calls itself to solve a problem by breaking it into smaller subproblems. Every recursive function must have a base case (a condition that stops the recursion) and a recursive case (where the function calls itself with a simpler input).

Without a base case, the function will call itself infinitely and cause a RecursionError.

Recursion is commonly used to solve problems like factorial, Fibonacci series, and countdown timers.

Lab Work:

Task 1:

```
def fibonacci(n):
    if n <= 0:
        return 0
    elif n ==1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
print("The fibonacci is :", fibonacci(3))
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\W
The fibonacci is : 2
```

Task 2:

```
def show(n):
    if n == 0: #base case
        return
    if n > 0:
        print(n)
        show(n-1)
show(0)
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\Week-13\.venv
Process finished with exit code 0
```

Task 3:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n-1)
print("The factorial is : ", factorial(5))
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\W
The factorial is : 120
```

Task 4:

```
def fibonacci(n):
    if n <= 0:
        return 0
    elif n ==1:
```

```
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
print("The fibonacci is :", fibonacci(3))
```

Result:

```
C:\Users\Fizzzzzz\PycharmPr
The fibonacci is : 2
```

Task 5:

```
def information(name,age,department,university):
    print("Name: " +name)
    print("Age: " + str(age))
    print("Department: " + department)
    print("University: " + university)
    return name, age, department, university
information("Fizza", 19, "Computer Engineering", "UET")
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\Wee
Name: Fizza
Age: 19
Department: Computer Engineering
University: UET
```

Conclusion:

This lab helped me understand how recursion works in Python. By implementing recursive programs for countdown, factorial, and Fibonacci, I learned how every recursive function needs a base case to stop the recursion and a recursive case that reduces the problem step by step until the base case is reached.

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 11	CLOs to be covered: CLO4
Lab Title: Exception Handling in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 11

Lab Goals/Objectives:

- To understand the concept of exceptions and errors in Python
- To learn how to use try and except blocks to handle exceptions
- To understand the difference between specific and general exception handling
- To learn how to write robust programs that do not crash on invalid input

Equipment Required: Computer system with PyCharm IDE and Python package installed on it.

Theory:

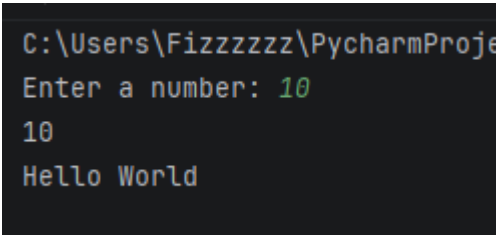
An exception is an error that occurs during the execution of a program and disrupts the normal flow of instructions. Python provides the try/except block to handle exceptions gracefully without crashing the program. Code that might raise an error is placed inside the try block, and the corresponding handler is written in the except block. Specific exceptions like ValueError and ZeroDivisionError can be caught individually, or a general Exception can be used to catch any error. Code outside the try/except block continues to execute regardless of whether an exception occurred.

Lab Wor:

Task 1:

```
try:
    a = int(input("Enter a number: "))
    print(a)
except ValueError:
    print("Invalid input.")
print("Hello World")
```

Result:



```
C:\Users\Fizzzzzz\PycharmProje
Enter a number: 10
10
Hello World
```

Task 2:

```
try:
    a = int(input("Enter a number: "))
    print(a)
except Exception as e:
    print("An error occured:", e)
print("Hello World")
```

Result:

```
C:\Users\Fizzzzzz\PycharmPro
Enter a number: 15
15
Hello World
```

Conclusion:

This lab helped me understand how exception handling works in Python. By using try/except blocks, I learned how to prevent programs from crashing on invalid input, how to catch specific errors like ValueError, and how to use the general Exception class to handle any unexpected error gracefully.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 12	CLOs to be covered: CLO2
Lab Title: Lambda functions in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.

Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 12

Lab Goals/Objectives:

- To understand the concept of lambda (anonymous) functions in Python
- To learn how to define and use lambda functions for simple operations
- To understand how to use lambda functions with map(), filter(), and reduce()
- To compare lambda functions with regular user-defined functions

Equipment Required: Computer system with PyCharm IDE and Python package installed on it

.

Theory:

A lambda function in Python is a small, anonymous function defined in a single line using the lambda keyword. Unlike regular functions defined with def, lambda functions do not have a name and can only contain a single expression. The syntax is: lambda parameters: expression. Lambda functions are commonly used with higher-order functions like map(), filter(), and reduce() which apply a function to elements of a list or sequence.

Lab Work:

Task 1:

```
def square(n):
    print(n**2)
    return n **2
square(5)
```

Result:

```
C:\Users\Fizzzzzz\Pycharm
25
```

Task 2:

```
a = lambda x: x*x
print(a(5))
```

Result:

```
C:\Users\Fizzzzzz\PycharmProject
25
```

Task 3:

```
a = lambda x,y,z : x+y+z
print(a(7,11,12))
```

Result:

```
C:\Users\Fizzzzzz\PycharmProject
30
```

Task 4:

```
a = [[1,2,3,4]]
s = list(map(lambda x: x*x,a))
print(s)
```

Result:

```
a = [[1,2,3,4]]
s = list(map(lambda x: x*x,a))
print(s)
```

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 13	CLOs to be covered: CLO2
Lab Title: MAP, Filter and Reduce in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 13

Lab Goals/Objectives:

To understand the concept of higher-order functions in Python

- To learn how to use map() to apply a function to every element of a list
- To learn how to use filter() to extract elements that satisfy a condition
- To learn how to use reduce() to combine elements of a list into a single value

Equipment Required: Computer system with PyCharm IDE and Python package installed on it

.

Theory:

Python provides three built-in higher-order functions: map(), filter(), and reduce(). The map() function applies a given function to every element of an iterable and returns a map object. The filter() function filters elements of an iterable based on a function that returns True or False. The reduce() function, available in the functools module, applies a function cumulatively to the elements of a sequence, reducing it to a single value. These functions are often used with lambda functions for concise, functional-style programming.

Lab Work:

Task 1:

```
a = [[1,2,3,4]]
s = list(map(lambda x: x*x,a))
```

```
print(s)
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\week_14\.venv\Scripts\python.exe "C:\Users\Fizzzzzz\PycharmProjects\week_14\task 4.py"
Traceback (most recent call last):
  File "C:\Users\Fizzzzzz\PycharmProjects\week_14\task 4.py", line 2, in <module>
    s = list(map(lambda x: x*x,a))
  File "C:\Users\Fizzzzzz\PycharmProjects\week_14\task 4.py", line 2, in <lambda>
    s = list(map(lambda x: x*x,a))
      ~^~
TypeError: can't multiply sequence by non-int of type 'list'
```

Task 2:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print("Even numbers:", evens)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProjects\week_14\task 2.py"
Even numbers: [2, 4, 6, 8, 10]
```

Task 3:

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print("Product:", product)
```

Result:

```
"C:\Users\Fizzzzzz\PycharmProjects\week_14\task 3.py"
Product: 120
```

Conclusion:

This lab helped me understand the three key higher-order functions in Python: map(), filter(), and reduce(). I learned how map() transforms every element, filter() selects elements based on a condition, and reduce() combines all elements into a single result. These functions make code more concise and readable when combined with lambda functions.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 14	CLOs to be covered: CLO2
Lab Title: OS Library, if __name__ == "__main__", is vs == in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.

Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 14

Lab Goals/Objectives:

- To understand how to use the OS library to interact with the file system
- To learn the use of **name == 'main'** to control script execution
- To understand global and local variables and how to access them
- To learn how to read from and write to files using file handling

Equipment Required: Computer system with PyCharm IDE and Python package installed on it

Theory:

The os module in Python provides functions to interact with the operating system, such as creating directories with `os.mkdir()`. The special variable **name** is set to **'main'** when a script is run directly, and to the module name when it is imported — this allows code to be conditionally executed. File handling in Python uses `open()` with modes 'r' (read), 'w' (write/overwrite), and 'a' (append). Global variables are accessible inside functions, and the `globals()` function returns a dictionary of all global variables

Lab Work:

Task 1:

```
import os
os.mkdir("Fizzup1")
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\week_14\
Process finished with exit code 0
```

Task 2:

```
import os
for i in range(0, 100):
    os.mkdir(f"day {i}")
```

Result:

No output is displayed.

Folders named day 0, day 1, day 2, ..., day 99 are created successfully.

Task 3:

```
a = open("FizzaFaisal.txt", "r")
s = a.read()
print(s)
a.close()
```

Result:

```
C:\Users\Fizzzzzz\PycharmProjects\week_14\
Hi, Fizza FaisalHi, Fizza Faisal
```

Task 4:

```
a = open("FizzaFaisal.txt", "a")
s = a.write("Hi, Fizza Faisal")
print(s)
a.close()
```

Result:

```
C:\Users\Fizzzzzz\PyCharm
16
```

Task 5:

```
a = open("FizzaFaisal.txt", "w")
s = a.write("Hi, Fizza")
print(s)
a.close()
```

Result:

```
C:\Users\Fizzzzzz\Pycharm
9
```

Task 6:

```
a = open("FizzaFaisal.txt", "r")
s = a.read()
print(s)
a.close()
```

Result:

```
C:\Users\Fizzzzzz\PycharmProj
Hi, Fizza
```

Task 7:

```
def welcome():
    print("Welcome to the class")
if __name__ == "__main__":
    welcome()
def goodbye():
    print("Goodbye to you")
goodbye()
```


Result:

```
C:\Users\Fizzzzzz\PycharmProjects\  
Welcome to the class  
Goodbye to you
```

Task 8:

```
import fizza  
fizza.welcome()  
fizza.goodbye()
```

Result:

```
C:\Users\Fizzzzzz\PycharmPr  
Welcome!  
Goodbye!
```

**University Of Engineering and Technology, Lahore
Computer Engineering Department**

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 30 June 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 15	CLOs to be covered: CLO3
Lab Title: GUI using Qt Designer in Python	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(10)						
Total						/10

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	9-10	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	7-8	L2	Submitted the lab tasks but have good understanding
Good	5-6	L3	Submitted the lab tasks but have weak understanding.
Basic	3-4	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1-2	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 15

Title: Musify App

Lab Goals/Objectives:

- To understand how to build a GUI application using PyQt5 in Python
- To learn how to use widgets like QMainWindow, QLabel, QPushButton, QSlider, and QListWidget
- To implement a media player using QMediaPlayer and QMediaContent
- To apply object-oriented programming concepts to create a real-world desktop application

Equipment Required: Computer system with PyCharm IDE, Python package, and PyQt5 library installed on it.

Theory:

PyQt5 is a Python library used to build desktop GUI (Graphical User Interface) applications. It provides a wide range of widgets such as buttons, labels, sliders, and list widgets that can be arranged to create interactive interfaces. QMainWindow serves as the main application window, while QMediaPlayer handles audio playback including play, pause, next, and previous controls. Qt Designer can be used to visually design the UI, which is then connected to Python logic to make the application functional.

Lab Work:

Task 1:

1. Main entry point (without this, the app won't run)

```
if __name__ == "__main__":  
    app = QApplication(sys.argv)  
    window = MusicPlayer()  
    window.show()  
    sys.exit(app.exec_())
```

2. Main class

```
class MusicPlayer(QMainWindow):
```

3. Constructor

```
def __init__(self):  
    super().__init__()
```

4. Media Player

```
self.player = QMediaPlayer()
```

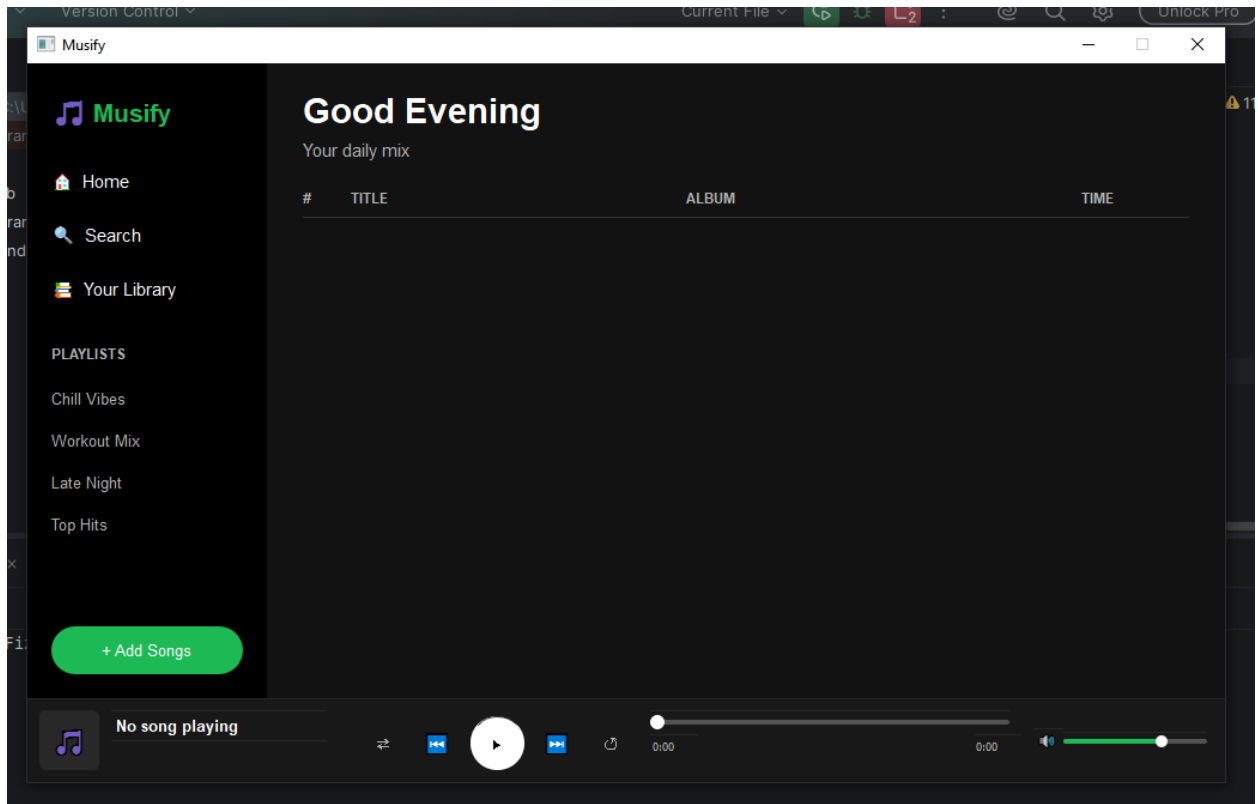
5. Signal Connections

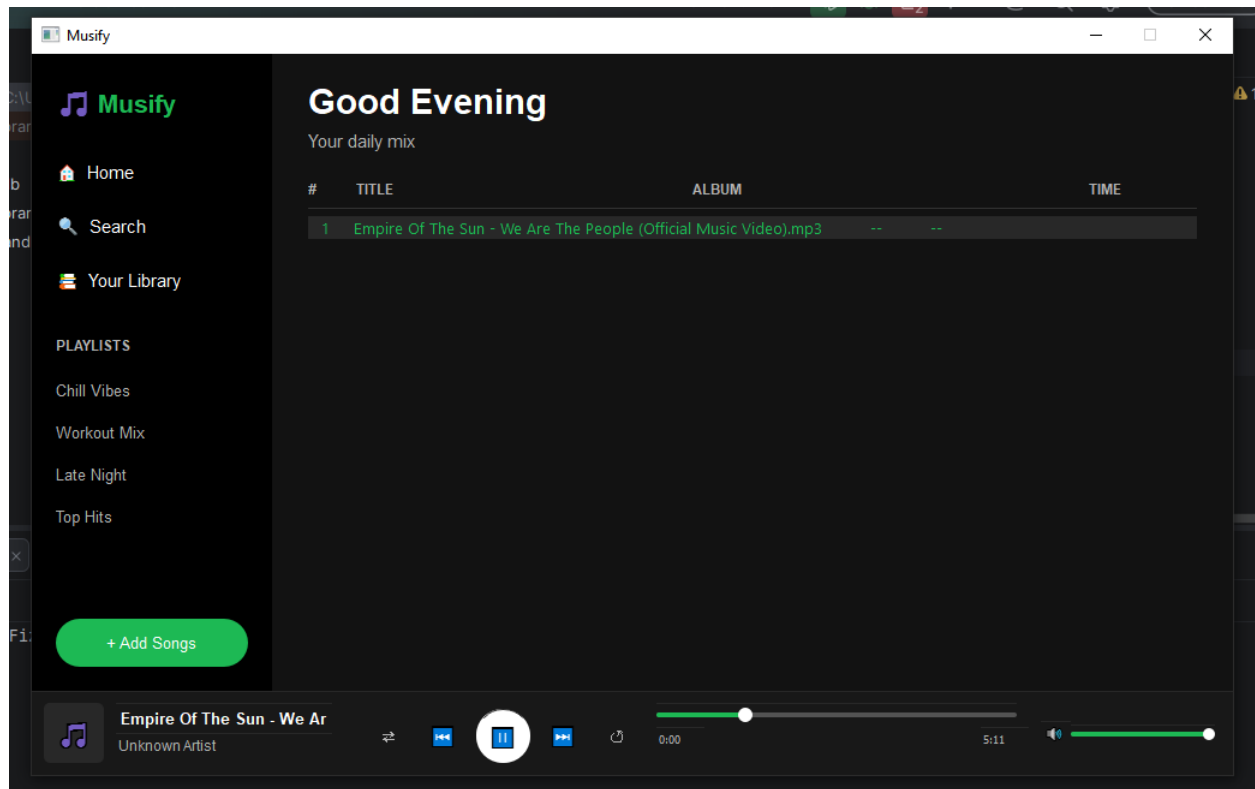
```
self.player.durationChanged.connect(self.update_duration)  
self.player.stateChanged.connect(self.update_play_button)
```

6. Building the Interface

```
self.build_sidebar()  
self.build_main_area()  
self.build_bottom_bar()
```

Result:





Qt Design:

